

Homework 1 心得報告

資訊三乙 11220105 楊耀甯

本次實作的五子棋(白方) AI，可以從以下幾點方面切入：

1. 如何在**探索深度(depth)**越大的情況下，讓程式能夠於 5 秒內將探索的動作做完。
2. 判斷在**探索深度(depth)**與**最大候選步數(max-candidates)**之間要如何取捨，才能有效做出最好的決策。
3. 將五子棋的各種棋面狀況，以優先重要程度由上往下 逐一判斷、累加分數：
 - a. 連棋狀況：五連(win)、活四(open four)、衝四(closed four)、活三(open three)、衝三(closed three)、活二(open two)、死二(closed two)、雙活四(open four number ≥ 2)、雙活三(open three number ≥ 2)、四三(open_four number ≥ 1 and open_three number ≥ 1)。
 - b. Position Heuristic：白棋將根據 neighbor_radius，把棋下在距離附近有其他子的地方，避免下在一個四周都沒有其他子的區塊。
 - c. 誘導黑棋禁手(Black Forbidden Trap)：除了 a、b 以外，將某點可能成為黑棋禁手的位置累加於白棋的分數中。
4. 基於 AI 所提供之基本 minimax 框架下，搭配 AI 協作針對候選棋步(candidate)排序以及最終棋面評分重新設計。

以上的幾點皆是藉由與生成式 AI(主要使用 Claude)協作之後，自己所發現的問題以及待改進之處，再次與 AI 討論並且給予適當 prompt 所得到之實作重點

一、與 AI 協作的過程

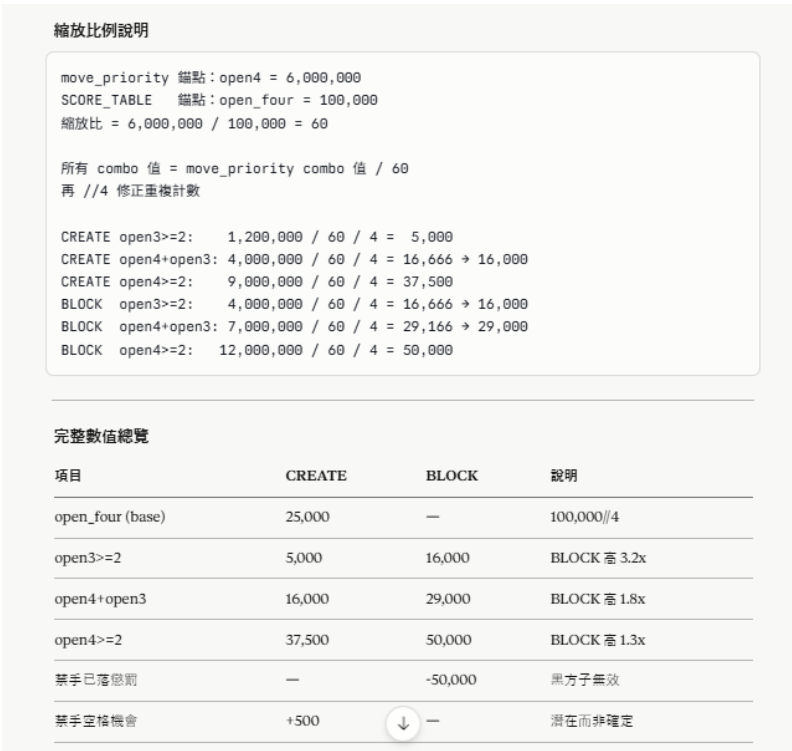
本次作業我認為最重要的點是要如何針對自己的需求下對 prompt，以本次實作的情況來說，AI 本身是擅長於「**提供程式的骨架**」，例如請 AI 寫好基於 minimax、DFS alpha-beta pruning 這些資料結構與演算法的程式框架，至於細節評分的部分，如上述第 3 點所提的連棋狀況、誘導禁手、Position Heuristic 等，由於 AI 在細節評分這塊往往只會把焦點放在 general case(如活四、活三、衝四等)而忽略如雙活四、雙活三等這些較複雜的棋路，因此必須額外將這些細節加以記錄 並提供好自己設計之基礎框架，再請 AI 加以修改、擴充，如下面的圖

一、二、三。

圖一：使用者輸入的需求 prompt。



圖二、圖三：AI 回覆片段。藉由 AI 分析，將需求保留、擴充、甚至是修改。



另外我認為本次實作最大的困難在於生成式 AI 無法精準測試程式效能，以我的程式為例，為了避免程式執行時間超過 5 秒，我特別於 minimax 開頭設立了 time_limit 的終止判斷條件(圖四)：

圖四：終止判斷條件，時間一旦超過 TIME_LIMIT 就直接 evaluate_board

```
if time.time() - _start_time > TIME_LIMIT:  
    return evaluate_board(board), -1, -1
```

這樣就會造成尚未選擇好最佳棋路之前就直接評估目前遍歷到的棋路，這樣就會造成深度達到一定的閥值時，在重要決策無法下在防堵的位置，讓對手贏棋(圖五)，因此為防止類似情況，我暫且將深度設為 5。

圖五： 白棋沒有防堵 4、5、7 這三顆黑子形成的活三，造成輸棋。

TIME_LIMIT = 4.9

WHITE: depth=6 max-candidate=2

BLACK: depth=5 max-candidate=5 (engine_minimax.py)



二、程式架構與 engine_minimax.py 相比新增之機制

首先，我的程式在 parse_args 方面完全承襲自 engine_minimax.py，因此在初始化深度以及最大候選步數的機制與 engine_minimax.py 是完全相同的，而由於我的程式是完全基於五子棋白方的立場進行設計，因此會與 engine_minimax.py 本身的通用設計有明顯落差(圖六)，

圖六：基於白子屬於防守多於進攻的策略，在後續偵測棋路時，為避免黑方可能的勝利，我將活四、活三、雙活四、雙活三、四三的黑棋分數設為大於白棋分數，其餘部分考量還是要適當進攻，因此白棋分數大於黑棋分。

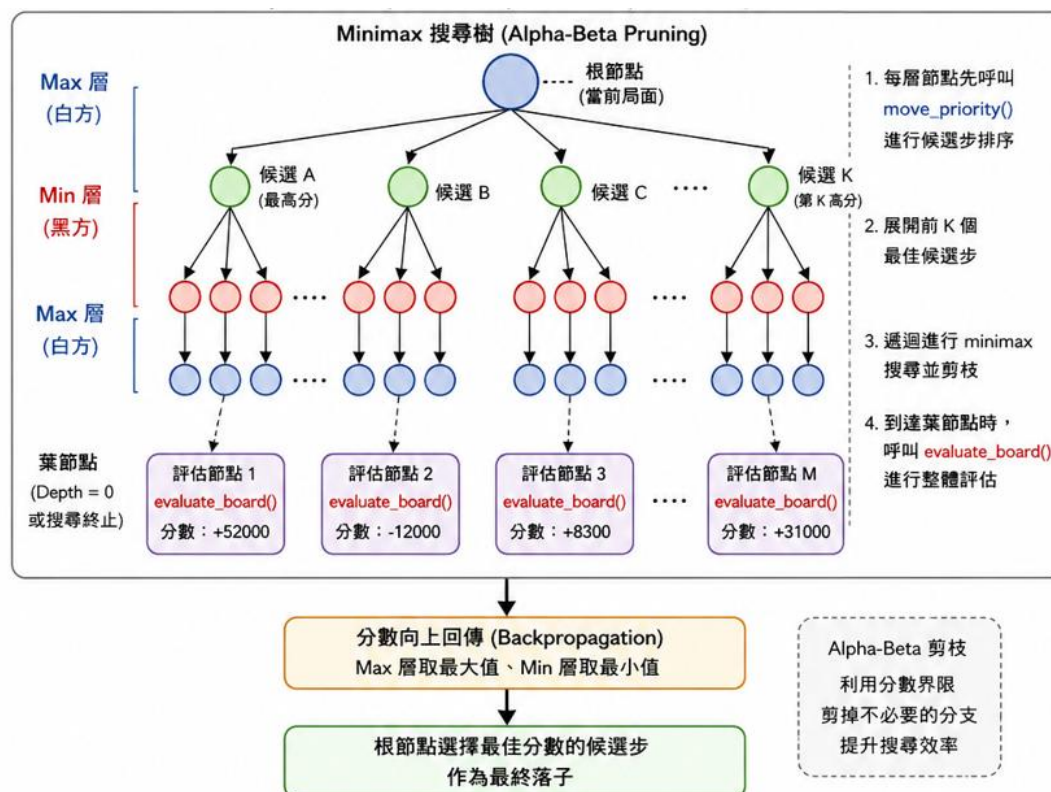
```
WHITE_WIN_SCORE = 100_000_000
BLACK_WIN_SCORE = 90_000_000
WHITE_OPEN_4 = 6_000_000
BLACK_OPEN_4 = 8_000_000
WHITE_CLOSED_4 = 1_500_000
BLACK_CLOSED_4 = 700_000
WHITE_OPEN_3 = 240_000
BLACK_OPEN_3 = 480_000
WHITE_CLOSED_3 = 60_000
BLACK_CLOSED_3 = 120_000
WHITE_OPEN_2 = 15_000
BLACK_OPEN_2 = 14_000
WHITE_CLOSED_2 = 4_000
BLACK_CLOSED_2 = 4_000

WHITE_MULTIPLE_OPEN_4 = 9_000_000
WHITE_4_3 = 4_000_000
WHITE_MULTIPLE_OPEN_3 = 1_200_000
BLACK_MULTIPLE_OPEN_4 = 12_000_000
BLACK_4_3 = 7_000_000
BLACK_MULTIPLE_OPEN_3 = 4_000_000
```

此外，不同於 engine_minimax.py 的 negamax，我的程式採用傳統的 **minimax** 搜尋棋路，理由是較為直觀易懂，至於我的程式關鍵決策評分系統可分為兩大函數：**move_priority()**與 **evaluate_board()**。

其運作狀態架構大致如圖七。

圖七：運作狀態架構圖(ChatGPT generated, 圖中兩個函數皆承襲自 engine_minimax.py，只是差別在於內部 implementation 細節的判斷條件新增)



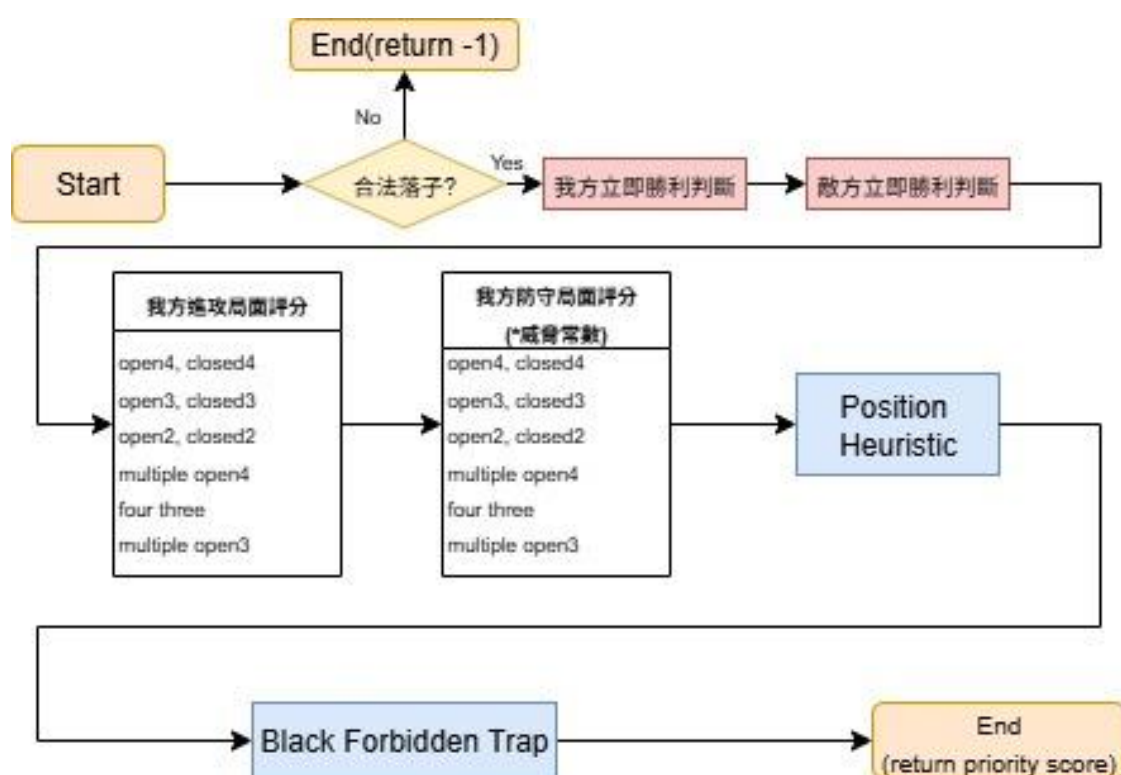
在 minimax 搜尋樹中，`move_priority()` 主要應用於每層節點展開前的候選棋步排序階段。當前棋盤狀態進入某一層搜尋時，程式會先透過 `ordered_move()` 呼叫 `move_priority()`，快速評估各合法棋步的即時戰術價值，例如是否能直接取勝、阻擋對手、形成活四、活三、雙活四或四三等高威脅局面，同時納入 Position Heuristic（中心位置、鄰近棋子）與 Black Forbidden 陷阱考量。根據評分結果，系統將候選棋步依分數排序並保留前 max-candidates 個最佳分支進一步展開，使 minimax 搜尋樹能優先搜尋高價值節點，進而提升 alpha-beta pruning 的剪枝效率。因此，`move_priority()` 屬於 minimax 搜尋樹前端的局部、戰術性節點排序工具，用來決定搜尋樹每層的展開順序。

相較之下，`evaluate_board()` 則主要作用於 minimax 搜尋樹的葉節點，也就是搜尋深度到底層、時間限制或提前終止時的靜態盤面評估。當搜尋停止展開後，`evaluate_board()` 會對當前葉節點的整體棋盤進行全局分析，評估所有連線型態，如活四、衝四、活三等基本威脅，以及雙活三、雙活四、四三等多重交叉威脅，綜合判斷白黑雙方的戰略優劣。此函數所產生的分數將作為該葉節點的局勢值，並沿 minimax 搜尋樹向上回傳，供 Max 層與 Min 層進行最佳

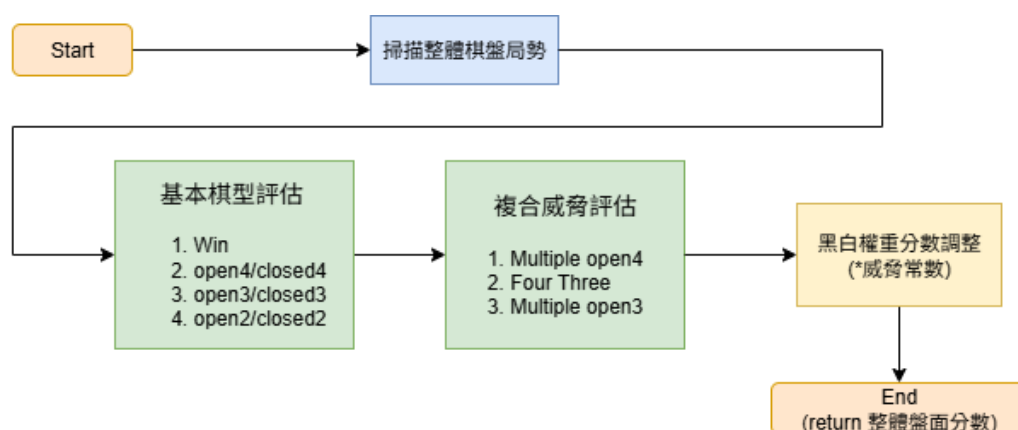
決策選擇。因此，`evaluate_board()` 屬於 `minimax` 搜尋樹底層的全局、戰略性評估函數，直接影響整棵搜尋樹的最終決策品質。

最後，與 `engine_minimax.py` 相比，`hw1_11220105.py` 在棋盤評估上更細緻。原版主要依據連線長度與開放端數量評估活四、活三、活二等基本型態，屬於通用且計算快速的 `heuristic`。我的程式則進一步區分活四、衝四、活三、衝三等情境，並加入雙活三、雙活四、四三連擊等高階複合威脅判斷，使 AI 更能掌握關鍵攻防局面。此外，白黑雙方採不同權重設計，提升策略真實性，並新增黑棋禁手陷阱判斷，強化白方優勢利用(圖八)(圖九)。

圖八：`move_priority` 函數流程圖(作者繪製，非 AI generated)。



圖九：`evaluate_board` 函數流程圖(作者繪製，非 AI Generated)。



三、結論

本次作業我會認為**最重要的關鍵還是在於 heuristic conditions 的設置**，像是除了活四、活三等這些基礎 heuristic 之外，還可以導入雙活四、雙活三、Position Heuristic、甚至禁手的概念，並利用 minimax 搜尋樹，以及好的配分方式，來達成最終的贏棋。

然後我認為不足的地方是這次作業在 **time limit 的處理比較偏向使用暴力法**，也就是在搜尋完成前，一旦超時就立刻停止並立即 evaluate 當前的盤面，造成樹的探索深度在 ≥ 6 之後容易有極端的錯誤發生，因此我認為這是最需改進之處，還有**最初在設計時並未考量到對手的強度**，由於 engine_minimax.py 的初始深度只有 2，造成我前面很長一段時間都以此基準來測試我的程式，然而一旦 engine_minimax.py 的深度加深，其進攻強度就會變得更加厲害，造成我的程式幾乎都在防守，最終落敗，對此我會認為這些也是可以加強的部分。